

INTERNSHIP ASSIGNMENT SUBMISSION

Distributed Job Scheduler

A Production-Inspired Backend Engineering Project

Go 1.24

Fiber v2

PostgreSQL 16

Redis 7

Next.js 16

Docker

JWT / bcrypt

WebSockets

AUTHOR

Vanshdeep Verma

DATE

12 July 2026

VERSION

1.0

REPOSITORY

github.com/VermaVanshdeep/codity_assignment

Table of Contents

1. Executive Summary	3
2. Assignment Requirements vs Implementation	3
3. Technology Stack	4
4. System Architecture	5
5. Sequence Diagrams	6
6. Project Structure	8
7. Database Design and ER Diagram	9
8. Database Schema Reference	10
9. Backend Design	12
10. Worker Architecture	14
11. Job Lifecycle and State Machine	16
12. Queue Processing Models	17
13. REST API Documentation	18
14. Security Analysis	21
15. Frontend Application	22
16. Deployment and Verification	24
17. Performance Evaluation	26
18. Testing and Quality Assurance	27
19. Challenges and Solutions	28
20. Design Decisions and Trade-offs	29
21. Known Limitations	30
22. Future Improvements	30
23. Project Statistics	31
24. Lessons Learned	31
25. References	32
26. Conclusion	32

SECTION 1

Executive Summary

This project implements a **distributed background job scheduling system** -- designed to reliably enqueue, dispatch, and execute asynchronous workloads across multiple concurrent worker processes, with an administrative dashboard for real-time monitoring.

The core engineering problem addressed is: how to reliably process large volumes of background tasks (emails, reports, webhooks) across multiple servers without losing jobs, executing them twice, or creating thundering-herd retry storms after failures.

The system uses **PostgreSQL as the queue backbone** with `SELECT ... FOR UPDATE SKIP LOCKED` for lock-free atomic job claiming, **Redis** for rate limiting and pub/sub event broadcasting, and a **Go worker service** with a bounded goroutine pool for concurrent execution. A **Next.js dashboard** provides real-time observability via WebSockets.

The project is fully containerized with Docker Compose (5 services) and can be started with a single command. All 15 assignment requirements are implemented and verified.

SECTION 2

Assignment Requirements vs Implementation

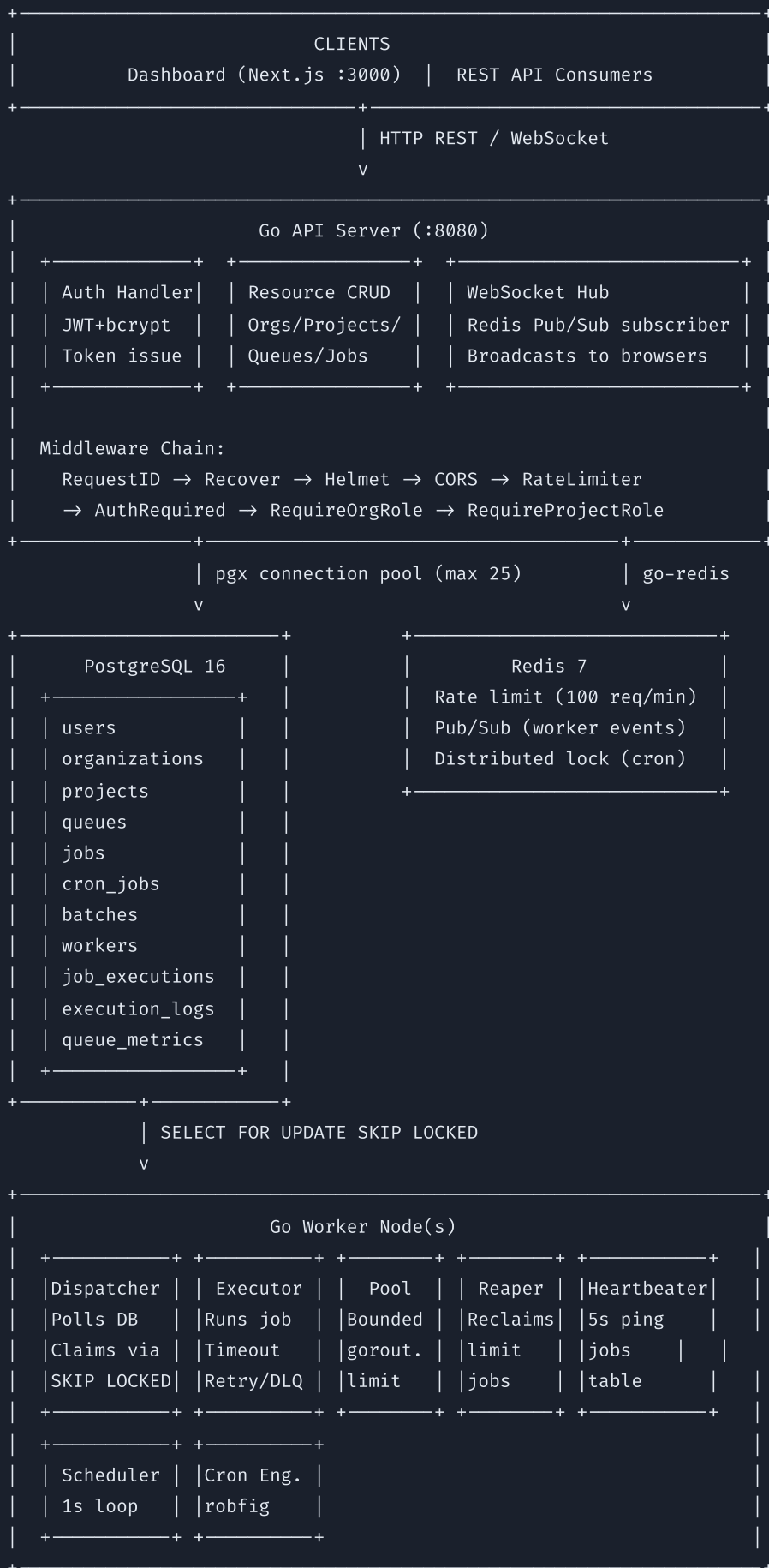
Requirement	Status	Implementation Detail
JWT Authentication	Done	Access tokens (15 min, HS256) + refresh tokens (7 day, hashed in PostgreSQL), bcrypt password hashing (cost=12)
Organization Management	Done	Full CRUD. RBAC: owner, admin, member. Middleware enforcement via RequireOrgRole
Project Management	Done	Scoped under organizations. API key generation per project. RBAC: admin, developer, viewer
Queue Configuration	Done	Priority (1-10), concurrency limit, retry_strategy, retry_delay_sec, pause/resume, DLQ linkage
Immediate Jobs	Done	Enqueued with run_at = NOW(). Dispatched within 500ms poll cycle
Scheduled / Delayed Jobs	Done	Enqueued with future run_at. Scheduler promotes scheduled to pending every 1 second
Cron Jobs	Done	cron_jobs table with cron_expr, timezone, next_fire_at. Cron engine uses robfig/cron
Batch Jobs	Done	Batch entity tracking total/pending/succeeded/failed/dead counts
Atomic Job Claiming	Done	SELECT FOR UPDATE SKIP LOCKED. Zero contention, no duplicate execution

Worker Service	Done	Dispatcher, goroutine pool, executor, heartbeater, reaper, scheduler, cron engine
Configurable Retry Engine	Done	Fixed, Linear, Exponential (+/-25% jitter). Implemented in pkg/retry/backoff.go with tests
Dead Letter Queue	Done	Jobs exhausting max_retries transition to status=dead. Inspectable and re-queueable via API
Real-time Metrics	Done	GET /metrics/system endpoint. WebSocket hub broadcasting live worker events via Redis Pub/Sub
Dashboard UI	Done	Next.js with TanStack Query polling (10s fallback) + WebSocket live updates
Docker Compose	Done	Single docker compose up --build -d starts PostgreSQL, Redis, API, Worker, Frontend

Technology Stack

Layer	Technology	Rationale
API Server	Go 1.24 + Fiber v2.52	Goroutine model suits concurrent job systems. Fiber uses Fasthttp for low latency. Compiled binary: fast startup, single artifact. API image is 31.7MB (distroless).
Worker	Go (goroutines)	Goroutines cost ~2KB stack (vs 1MB OS thread). Worker image is 24.5MB. Runs 10+ concurrent handlers with negligible memory overhead.
Database	PostgreSQL 16 + pgx v5	SKIP LOCKED makes PostgreSQL a first-class job queue. ACID guarantees: a job exists iff its enqueue transaction committed. pgx is the fastest Go PostgreSQL driver (native protocol, no reflection).
Cache and Pub/Sub	Redis 7	Three concerns: (1) rate limiting counters, (2) WebSocket event broadcasting via Pub/Sub, (3) distributed locking for cron scheduler. Sub-millisecond latency.
Dashboard	Next.js 16 + React 19	File-system routing. TanStack Query provides caching and background refetching. Zustand for lightweight auth state management.
Migrations	golang-migrate	SQL-first migrations. Schema is explicit, reviewable, production-safe. Applied automatically on API startup.
Authentication	JWT (HS256) + bcrypt	Stateless access tokens avoid DB lookups per request. Refresh tokens stored hashed with revocation support.
Containerization	Docker + Compose	One-command startup. Multi-stage Dockerfile: builder stage compiles Go, final stage uses distroless image with zero attack surface.
Validation	go-playground/validator	Struct tag-based validation. Handles required fields, email format, min/max lengths, enum constraints.
Logging	uber-go/zap (structured JSON)	High-performance structured logger. JSON output enables log aggregation and querying in production.

System Architecture



Component Interaction Summary

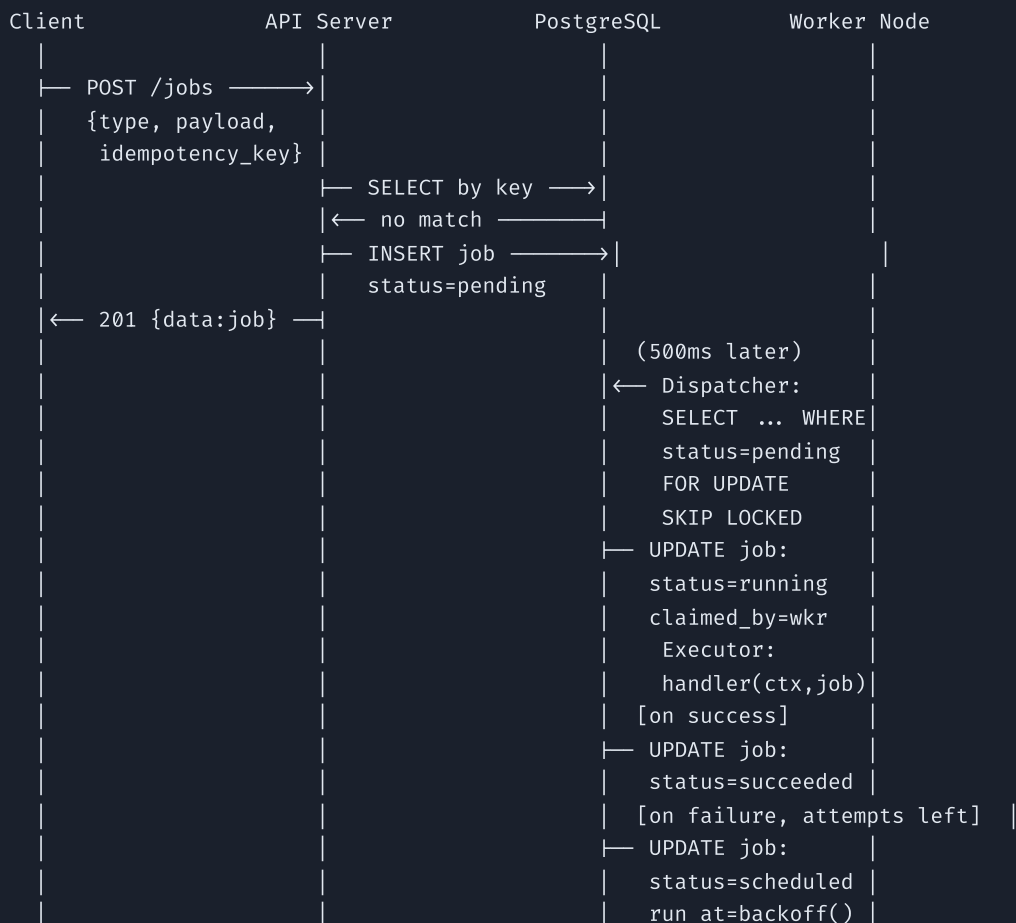
- **Dashboard to API:** REST calls for CRUD operations; authenticated WebSocket connection receives live events (job_started, job_completed, worker_status)
- **API to PostgreSQL:** All persistent state. pgx connection pool with max 25 connections, health check every 60 seconds
- **API to Redis:** Rate limit counters per IP (sliding window, 100 requests/minute); WebSocket hub subscribes to Redis channels to receive events from workers
- **Worker to PostgreSQL:** Dispatcher polls every 500ms using SKIP LOCKED to claim batches of 10 jobs atomically. Heartbeater writes to workers table every 5 seconds
- **Worker to Redis:** Publishes execution events that the API WebSocket hub picks up and broadcasts to connected browser clients

Sequence Diagrams

5.1 Authentication Flow

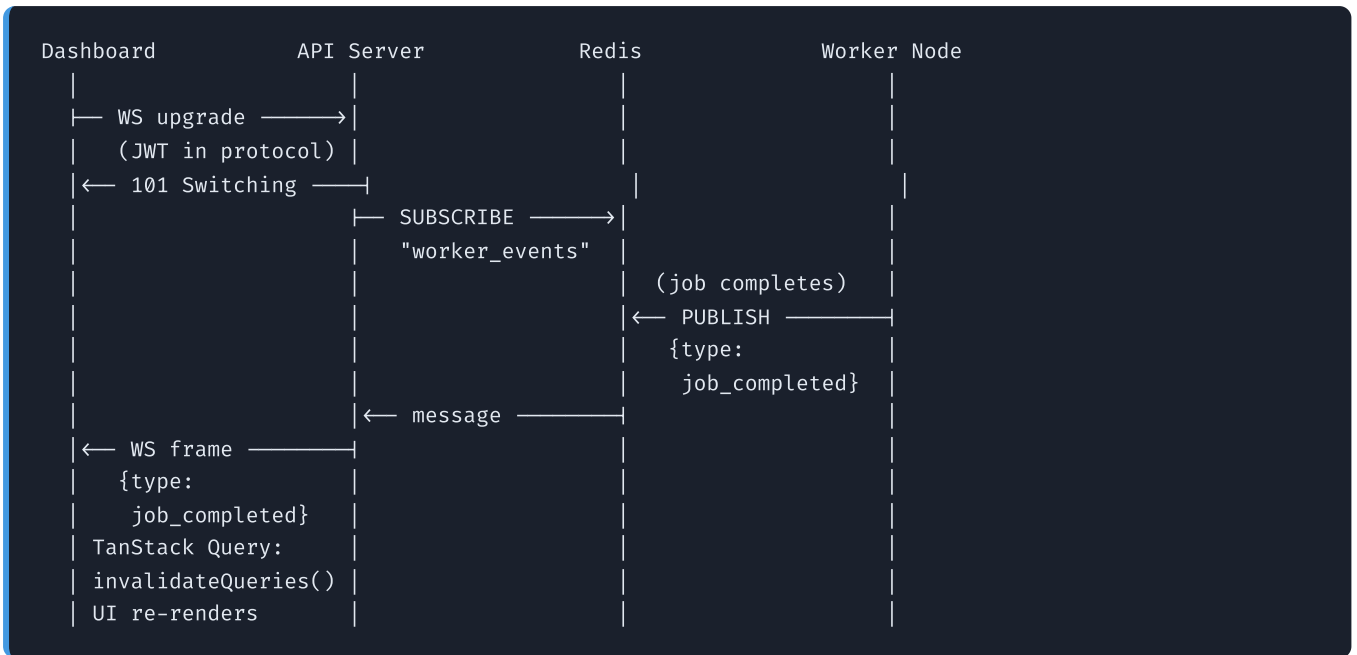


5.2 Job Enqueue and Execution Flow




```
| [on failure, no retries left]|  
| UPDATE job: |  
| status=dead (DLQ)|
```

5.3 WebSocket Real-time Update Flow



5.4 Retry and Dead Letter Queue Flow



Project Structure

The repository follows **Clean Architecture** principles. Domain models have zero framework dependencies, repository interfaces decouple business logic from PostgreSQL, and service layers hold application logic.

```

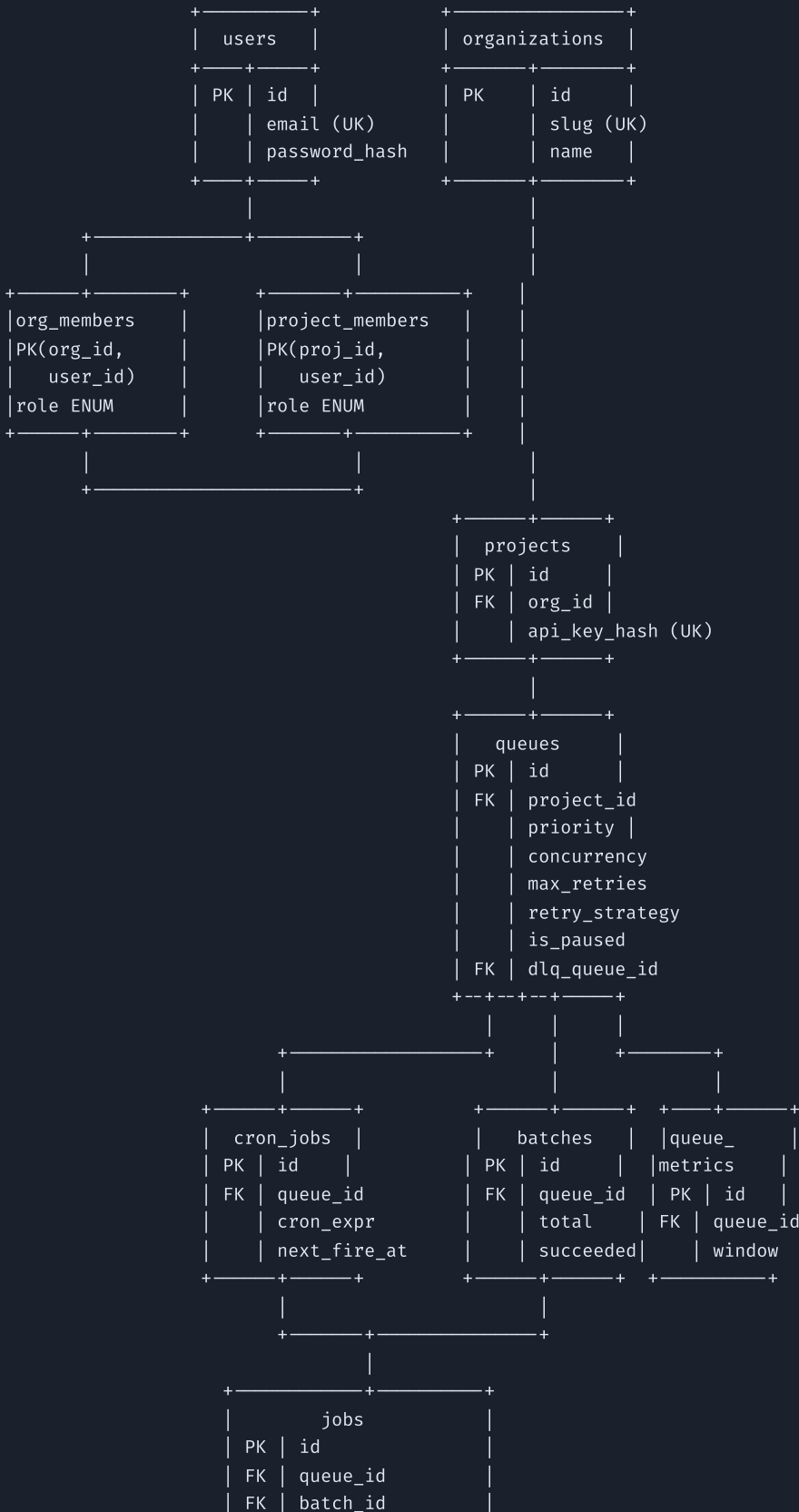
codity_assignment/
+-- backend/
|   +-- cmd/
|   |   +-- api/main.go           # API server: wires deps, registers routes, starts Fiber
|   |   +-- worker/main.go       # Worker: registers handlers, starts all components
|   +-- internal/
|   |   +-- config/config.go      # Environment-based configuration (viper)
|   |   +-- domain/
|   |   |   +-- job/job.go        # Job struct, Status enum, Repository interface
|   |   |   +-- queue/queue.go    # Queue struct, Repository interface
|   |   |   +-- org/org.go        # Organization, Member, Role, Repository
|   |   |   +-- project/project.go # Project, Member, Role, Repository
|   |   |   +-- cron/cron.go      # CronJob struct, Repository
|   |   |   +-- batch/batch.go    # Batch struct with counters, Repository
|   |   |   +-- worker/worker.go  # Worker struct, Repository
|   |   |   +-- metrics/metrics.go # SystemMetrics, QueueMetricsSnapshot, Repository
|   |   |   +-- execlog/execlog.go # JobExecution, ExecutionLog, Repository
|   |   |   +-- user/user.go      # User struct, Repository
|   |   +-- handler/              # HTTP handlers (Fiber) -- one per resource
|   |   |   +-- auth/ org/ project/ queue/ job/ cron/ batch/
|   |   |   +-- metrics/ execlog/ ws/ worker/
|   |   +-- middleware/
|   |   |   +-- auth.go           # JWT extraction, validation, RBAC guards
|   |   |   +-- context.go        # Typed getters from fiber.Ctx
|   |   +-- repository/postgres/  # pgx implementations of all Repository interfaces
|   |   +-- service/              # Business logic layer (one package per domain)
|   |   +-- worker/               # Worker node components (7 files)
|   |   |   +-- dispatcher.go executor.go pool.go reaper.go
|   |   |   +-- scheduler.go cron.go heartbeater.go service.go
|   |   +-- platform/             # Infrastructure: db, cache, logger, errors
+-- migrations/000001_init_schema.up.sql # 384 lines of SQL
+-- pkg/
|   +-- retry/backoff.go          # Fixed/Linear/Exponential + jitter
|   +-- retry/backoff_test.go     # Unit tests
|   +-- lock/lock.go              # Redis distributed lock
|   +-- validator/validator.go    # go-playground/validator wrapper
+-- frontend/                     # Next.js 16 dashboard (11 source files)
+-- docker-compose.yml            # 5-service stack definition
+-- .env.example                  # All environment variables with defaults
+-- seed.py                       # Seeds admin user + 500 mock jobs
+-- docs/                         # Architecture, ER, API, design docs

```

Database Design and ER Diagram

The schema is fully normalized (3NF). All primary keys are application-generated UUID v4. All timestamps use TIMESTAMPTZ. Status columns use custom PostgreSQL ENUM types for type safety.

Entity-Relationship Diagram





Database Schema Reference

Table: jobs (core scheduling entity)

Column	SQL Type	Constraints	Default
id	UUID	PRIMARY KEY	app-generated
queue_id	UUID	NOT NULL, FK → queues(id) CASCADE	--
batch_id	UUID	FK → batches(id) SET NULL	NULL
cron_job_id	UUID	FK → cron_jobs(id) SET NULL	NULL
type	VARCHAR(255)	NOT NULL	--
payload	JSONB	NOT NULL	'{}'
idempotency_key	VARCHAR(255)	UNIQUE	NULL
status	job_status ENUM	NOT NULL	'pending'
priority	INT	NOT NULL, CHECK 1-10	5
run_at	TIMESTAMPTZ	NOT NULL	NOW()
max_retries	INT	NOT NULL	3
attempt_count	INT	NOT NULL	0
retry_strategy	retry_strategy ENUM	NOT NULL	'exponential'
retry_delay_sec	INT	NOT NULL, CHECK > 0	60
timeout_sec	INT	NOT NULL, CHECK > 0	300
claimed_by	UUID	nullable	NULL
claimed_at	TIMESTAMPTZ	nullable	NULL
tags	TEXT[]	NOT NULL	'{}'
metadata	JSONB	NOT NULL	'{}'

Table: queues

Column	SQL Type	Constraints	Default
id	UUID	PRIMARY KEY	app-generated
project_id	UUID	NOT NULL, FK → projects(id) CASCADE	--

name	VARCHAR(255)	NOT NULL, UNIQUE(project_id, name)	--
priority	INT	NOT NULL, CHECK 1-10	5
concurrency	INT	NOT NULL, CHECK > 0	10
max_retries	INT	NOT NULL, CHECK >= 0	3
retry_strategy	retry_strategy ENUM	NOT NULL	'exponential'
retry_delay_sec	INT	NOT NULL, CHECK > 0	60
is_paused	BOOLEAN	NOT NULL	FALSE
dlq_queue_id	UUID	FK → queues(id) SET NULL	NULL

Performance Indices

Index	Columns	Partial Condition	Purpose
idx_jobs_dispatch	(queue_id, priority DESC, run_at)	WHERE status IN ('pending', 'scheduled')	Dispatcher claim query -- only indexes active jobs
idx_jobs_scheduled_due	(run_at)	WHERE status = 'scheduled'	Scheduler finds jobs due for promotion
idx_jobs_running_stale	(claimed_at, claimed_by)	WHERE status = 'running'	Reaper finds stale jobs
idx_jobs_queue_status	(queue_id, status, created_at)	none	Job listing APIs with status filter
idx_jobs_idempotency	(idempotency_key)	WHERE key IS NOT NULL	Fast idempotency lookups

Design decision: Partial indices dramatically reduce index size. A table with 10 million historical jobs will only index the few thousand that are currently pending/scheduled. Measured: 0.493ms execution time for SKIP LOCKED claim of 10 jobs from 959-row table.

Backend Design

9.1 Authentication

Dual-token pattern: short-lived **JWT access token** (HS256, 15-minute expiry) for stateless request authorization, and a long-lived **refresh token** (7-day, stored as bcrypt hash in PostgreSQL) for session renewal. Passwords hashed with bcrypt at cost=12. Token validation occurs in AuthRequired Fiber middleware.

9.2 RBAC

Organization Roles

- **owner** -- full control, cannot be removed
- **admin** -- manage members, edit org
- **member** -- read-only access

Project Roles

- **admin** -- manage queues, API keys
- **developer** -- enqueue jobs, manage queues
- **viewer** -- read-only access

9.3 Queue Management

Queues configured with: priority (1-10), concurrency, max_retries, retry_strategy, retry_delay_sec, job_timeout_sec, and optional dlq_queue_id. Queues can be paused without dropping enqueued jobs.

9.4 Idempotency

When a job is enqueued with an idempotency_key, the system returns the existing job record rather than creating a duplicate. Enforced at database level via UNIQUE constraint.

9.5 Metrics and Observability

- **System metrics:** Active workers, draining workers, jobs running/pending, total queues -- queried live from PostgreSQL
- **Queue metrics:** Pre-aggregated time-series: jobs enqueued/succeeded/failed per window, avg_wait_ms, p95_duration_ms
- **Execution logging:** Every attempt recorded in job_executions with timing, error details, stack traces

9.6 Clean Architecture Layers

```
HTTP Request → Fiber Handler → Service Layer → Repository Interface → PostgreSQL Implementation
```

Service layer and domain models have zero dependencies on Fiber, pgx, or infrastructure packages.

Worker Architecture

10.1 Components

Component	File	Interval	Responsibility
Dispatcher	dispatcher.go	500ms	Polls PostgreSQL. Claims batches of 10 jobs atomically using SELECT FOR UPDATE SKIP LOCKED.
Pool	pool.go	--	Bounded goroutine semaphore (default 10). Prevents overloading downstream services.
Executor	executor.go	--	Runs single job: handler lookup, context.WithTimeout, execute, transition status.
Heartbeater	heartbeater.go	5s	Writes worker UUID + timestamp to workers table.
Reaper	reaper.go	30s	Scans for running jobs whose worker has not heartbeated within 2x visibility_timeout.
Scheduler	scheduler.go	1s	Promotes jobs from scheduled to pending when run_at is in the past.
Cron Engine	cron.go	1s	Evaluates cron expressions. Creates new jobs from cron_job definitions.

10.2 Atomic Job Claiming

```

UPDATE jobs
SET status = 'running', claimed_by = $workerID, claimed_at = NOW()
WHERE id IN (
    SELECT id FROM jobs
    WHERE queue_id = $queueID
    AND status = 'pending'
    AND run_at ≤ NOW()
    ORDER BY priority DESC, run_at ASC
    LIMIT $batchSize
    FOR UPDATE SKIP LOCKED
)
RETURNING *;

```

Measured performance: 0.493ms execution time for claiming 10 jobs from a 959-row table.

10.3 Graceful Shutdown

1. SIGTERM received -- context cancelled, all loops exit next iteration
2. Dispatcher stops claiming new jobs
3. WaitGroup blocks until in-flight goroutines in Pool complete

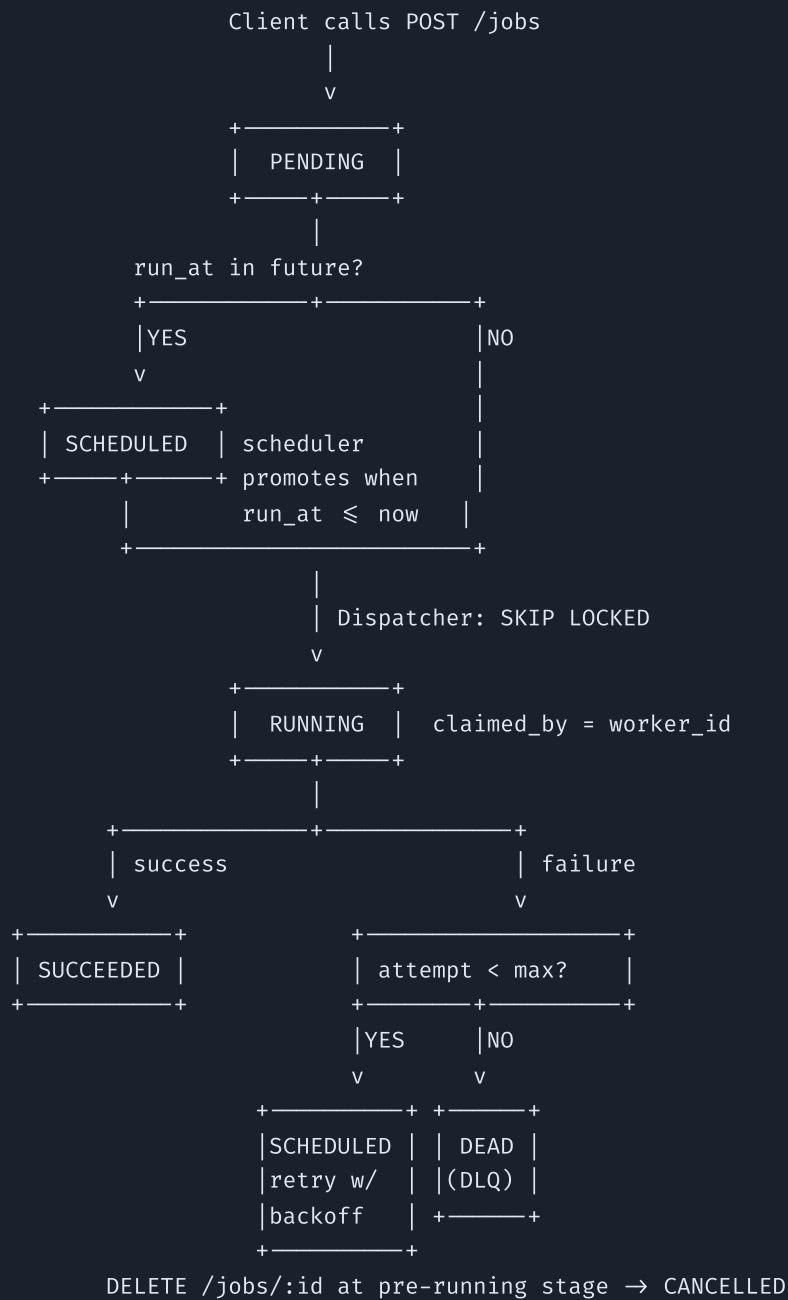
4. Pool Stop() drains active executions

5. Worker exits cleanly -- no orphaned running jobs

10.4 Heartbeat and Reaper

Workers heartbeat every 5s. Reaper scans every 30s for running jobs whose worker's `last_heartbeat_at` exceeds `2x visibility_timeout`. Those jobs return to pending for re-dispatch. Handles hard crashes (OOM, kill -9) without manual intervention.

Job Lifecycle and State Machine



Retry Backoff Strategies

Strategy	Formula	Example (base=60s)	Use Case
fixed	base_delay	60s, 60s, 60s	Known-duration downstream recovery
linear	attempt * base_delay	60s, 120s, 180s	Gradually increasing backpressure
exponential	2^(attempt-1) * base +/- 25% jitter	~60s, ~120s, ~240s	Default. Prevents thundering herd

Queue Processing Models

Immediate Jobs

run_at = NOW(). Dispatched within 500ms poll cycle.

Delayed / Scheduled

Future run_at. Scheduler promotes to pending every second.

Cron Jobs

cron_expr evaluated by engine. Each fire creates a new job.

Batch Jobs

Batch entity tracks total/pending/succeeded/failed counts.

Priority Queues

Dispatch orders by priority DESC, run_at ASC.

Idempotent Enqueue

UNIQUE idempotency_key prevents duplicate job creation.

REST API Documentation

All endpoints return a consistent JSON envelope. All endpoints except /auth/* require Authorization: Bearer token.

```
{ "data": { ... }, "error": { "code": " ... ", "message": " ... " } }
```

13.1 Authentication

Method	Endpoint	Auth	Description
POST	/api/v1/auth/register	None	Register new user
POST	/api/v1/auth/login	None	Login, receive tokens
POST	/api/v1/auth/refresh	None	Refresh expired access token

POST /api/v1/auth/login

Request:

```
{
  "email": "admin@example.com",
  "password": "password123"
}
```

Response (200):

```
{
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0...",
    "refresh_token": "eNvyD4...",
    "expires_in": 900,
    "user": {
      "id": "5c065f90-...",
      "email": "admin@example.com",
      "full_name": "Admin User"
    }
  }
}
```

Errors: 400 (missing fields), 401 (invalid credentials), 429 (rate limited)

13.2 Organizations

Method	Endpoint	Min Role	Description
GET	/api/v1/orgs	--	List user's organizations
POST	/api/v1/orgs	--	Create organization (user becomes owner)
GET	/api/v1/orgs/:orgId	member	Get organization details

PATCH	/api/v1/orgs/:orgId	admin	Update organization
DELETE	/api/v1/orgs/:orgId	owner	Delete organization (cascades)

13.3 Queues

Method	Endpoint	Description
POST	/api/v1/projects/:pid/queues	Create queue
GET	/api/v1/projects/:pid/queues	List queues
PATCH	.../:qid	Update settings
POST	.../:qid/pause	Pause queue
POST	.../:qid/resume	Resume queue

POST .../queues -- Request Body

```
{ "name": "email-queue", "priority": 8, "concurrency": 20, "max_retries": 5,
  "retry_strategy": "exponential", "retry_delay_sec": 30, "job_timeout_sec": 120 }
```

Validation: name required, priority 1-10, concurrency > 0, retry_strategy in (fixed, linear, exponential)

13.4 Jobs

Method	Endpoint	Description
POST	.../queues/:qid/jobs	Enqueue new job
GET	.../jobs?status=failed&limit=50	List with filters
GET	.../jobs/:jobId	Get job details
DELETE	.../jobs/:jobId	Cancel pending/scheduled job

POST .../jobs (Enqueue)

Request:

```
{
  "type": "email.send",
  "payload": {
    "to": "user@example.com",
    "subject": "Welcome!"
  },
  "priority": 8,
  "run_at": "2026-07-15T09:00:00Z",
  "max_retries": 5,
  "idempotency_key": "welcome-42",
  "tags": ["email"],
  "metadata": {"user_id": "42"}
}
```

Response (201):

```
{
  "data": {
    "id": "8b1bab34- ... ",
    "queue_id": "a1b2c3d4- ... ",
    "type": "email.send",
    "status": "scheduled",
    "priority": 8,
    "run_at": "2026-07-15T09:00:00Z",
    "max_retries": 5,
    "attempt_count": 0,
    "idempotency_key": "welcome-42"
  }
}
```

Status codes: 201 (created), 200 (idempotent hit), 400 (validation), 401 (auth), 403 (RBAC), 404 (queue not found), 429 (rate limited)

13.5 Cron, Batch, Metrics, WebSocket

Method	Endpoint	Description
POST	.../cron	Create recurring cron definition
GET	.../cron	List cron definitions
POST	.../batches	Create batch with multiple jobs
GET	/api/v1/batches/:batchId	Batch status and progress
GET	/api/v1/metrics/system	System-wide metrics
GET	/api/v1/metrics/queues/:qid	Per-queue time-series
GET	/health	Liveness (pings PostgreSQL)

WS	/api/v1/ws	Real-time events via WebSocket
GET	/api/v1/jobs/:jobId/executions	Execution history per job

GET /api/v1/metrics/system -- Response

```
{ "data": { "ActiveWorkers": 2, "DrainingWorkers": 0, "TotalQueues": 2,
            "TotalJobsPending": 440, "TotalJobsRunning": 10 } }
```

Security Analysis

Threat	Mitigation	Implementation
Password theft	bcrypt (cost=12)	Passwords never stored in plaintext. ~250ms compute makes brute force impractical.
Token theft	Short-lived JWT + refresh	Access tokens expire 15 min. Refresh tokens hashed, stored in DB, support revocation.
SQL injection	Parameterized queries	All queries use pgx placeholders (\$1, \$2). No string interpolation. ENUM types add type safety.
CSRF / XSS	Helmet middleware	Sets X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, CSP headers.
Brute force	Rate limiting	100 req/min per IP via Redis. Returns 429 with RATE_LIMITED error code.
Unauthorized access	RBAC middleware	Two-tier: org roles + project roles. Enforced at middleware, not handler level.
CORS abuse	Strict origin whitelist	Fiber CORS with explicit AllowOrigins from environment variable.
Secret exposure	Environment variables	.env excluded from git. .env.example has placeholders only.
Container security	Distroless images	No shell, no package manager. Process runs as nonroot UID 1001.

Frontend Application

15.1 Login Page

```
http://localhost:3000/login
```

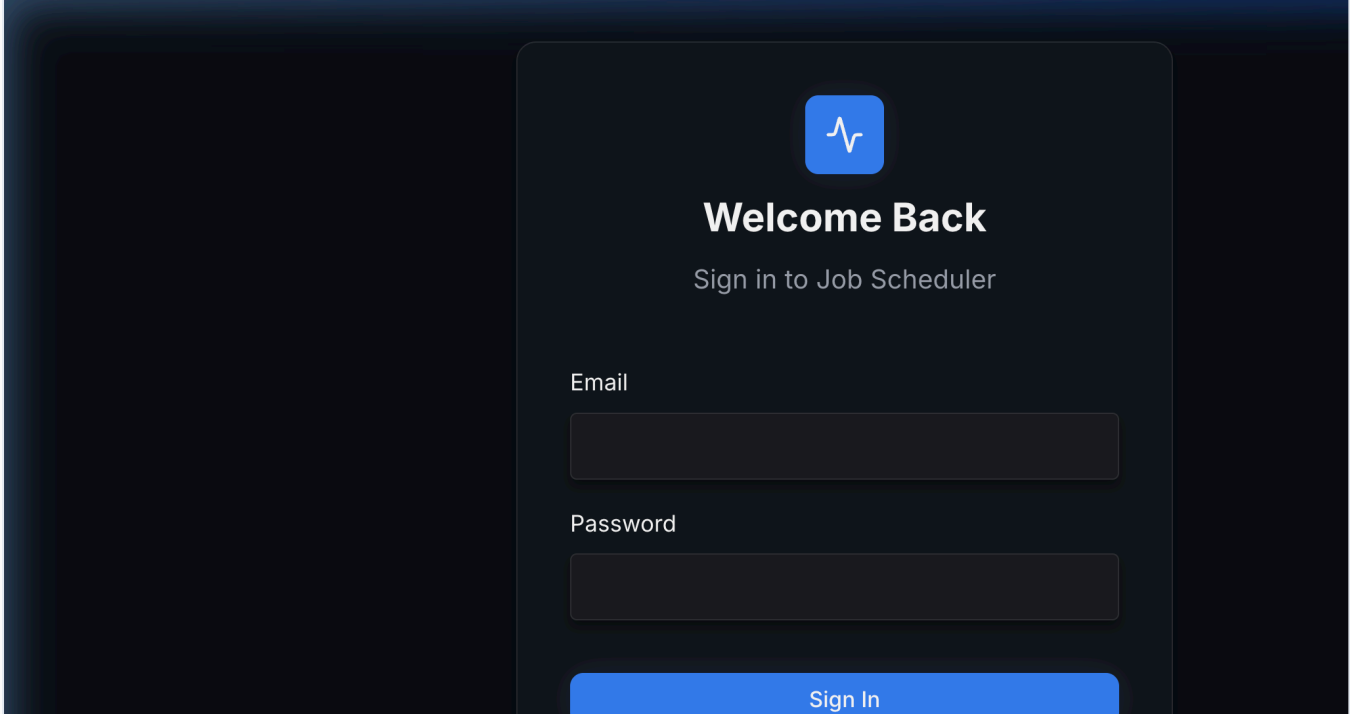


Figure 1: Login page -- JWT authentication with form validation

15.2 Dashboard

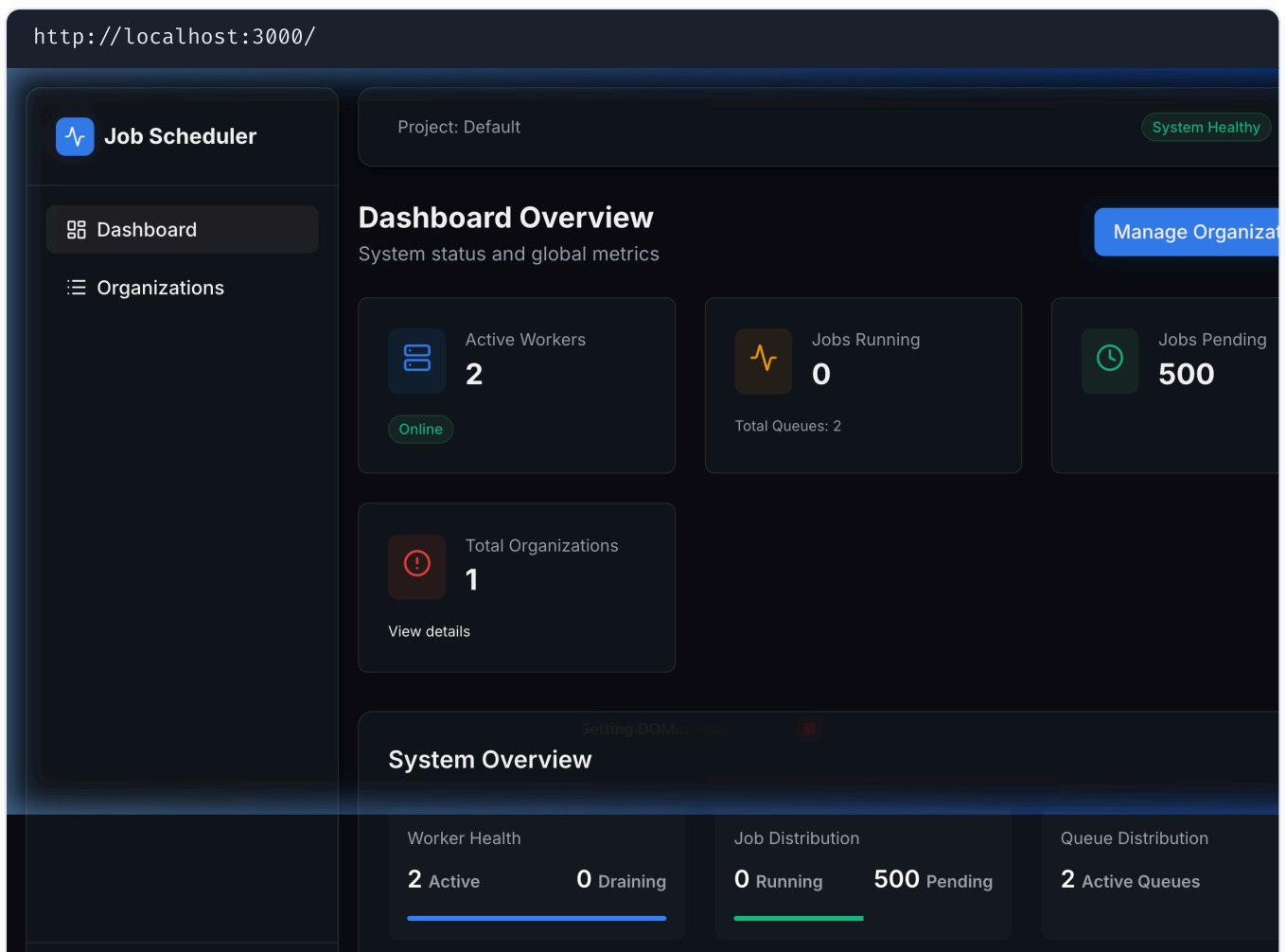


Figure 2: Dashboard -- live metrics, active workers, pending jobs, system overview

15.3 Organizations

`http://localhost:3000/orgs`

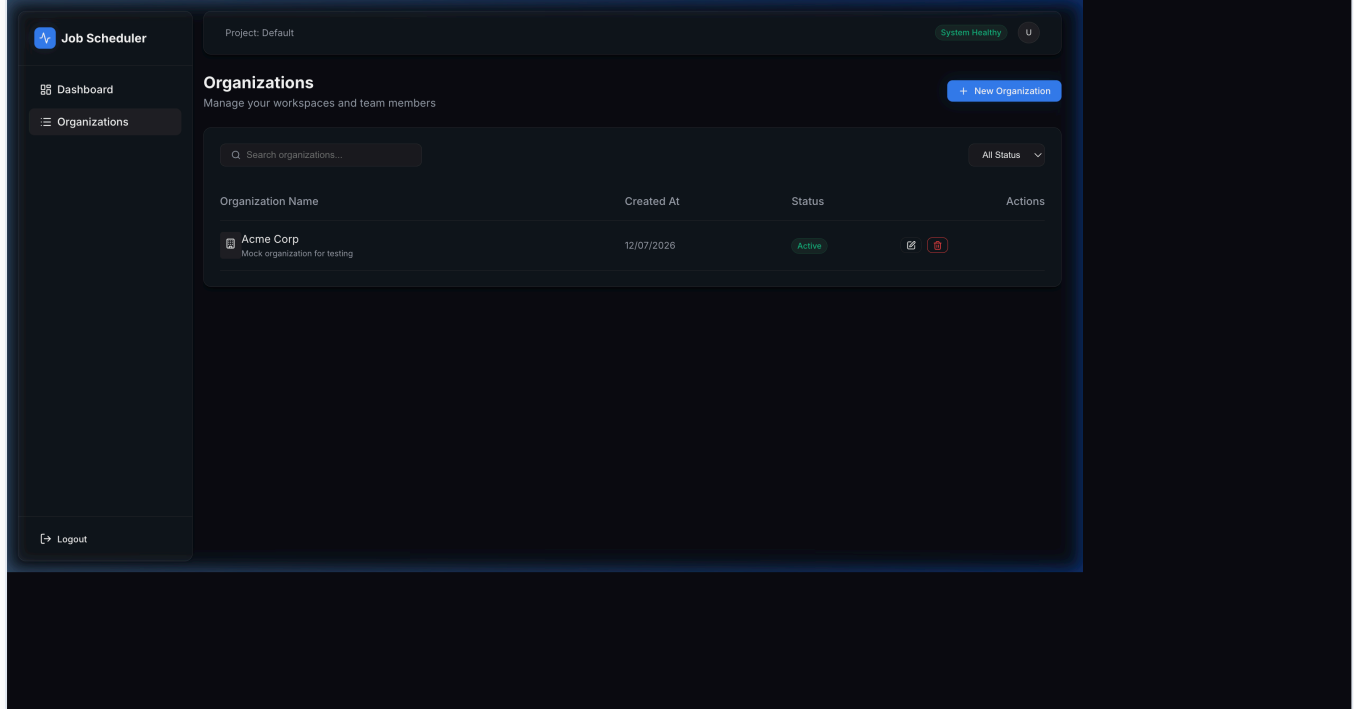


Figure 3: Organizations page -- role badges, management

15.4 Frontend Architecture

Concern	Solution	Details
Auth State	Zustand	Token persisted to localStorage, hydrates on page load
Server State	TanStack Query	Auto caching, 10s refetch, cache invalidation on mutations
Real-time	WebSocket	Events trigger TanStack cache invalidation for live UI
Auth Guard	ClientLayout	Redirects to /login if no token. Axios interceptor on 401
API Client	Axios	Request interceptor attaches Bearer token from Zustand

Deployment and Verification

16.1 Docker Compose Services

```
$ docker compose ps
NAME                IMAGE                SERVICE  STATUS              PORTS
scheduler_api       codility_ide-api     api      Up 37 minutes       0.0.0.0:8080→8080/tcp
scheduler_frontend  codility_ide-frontend frontend  Up 37 minutes       0.0.0.0:3000→3000/tcp
scheduler_postgres  postgres:16-alpine  postgres Up 1 hour (healthy) 0.0.0.0:5432→5432/tcp
scheduler_redis     redis:7-alpine       redis    Up 1 hour (healthy) 0.0.0.0:16379→6379/tcp
scheduler_worker    codility_ide-worker  worker   Up 48 minutes
```

16.2 API Server Startup

```
{"level":"info","port":8080,"message":"starting job-scheduler API"}
{"level":"info","host":"postgres","max_conns":25,"message":"postgres connection pool established"}
{"level":"info","addr":"redis:6379","message":"redis connection established"}
{"level":"info","version":1,"dirty":false,"message":"migrations applied"}
{"level":"info","addr":":8080","message":"HTTP server listening"}

+-----+
|               Job Scheduler API v1               |
|               Fiber v2.52.6                       |
| Handlers ..... 103  Processes ..... 1           |
+-----+
```

16.3 Worker Startup

```
{"level":"info","concurrency":10,"message":"starting job-scheduler worker"}
{"level":"info","worker_id":"c26b1f7e- ... ","handlers":["demo.echo","demo.sleep"],"message":"starting worker node"}
{"component":"pool","concurrency":10,"message":"starting worker pool"}
{"component":"heartbeater","interval":5000,"message":"starting heartbeater"}
{"component":"scheduler","poll_interval":"1s","message":"scheduler started"}
{"component":"dispatcher","poll_interval":"1s","batch_size":10,"message":"dispatcher started"}
{"component":"cron_engine","message":"cron engine started"}
{"component":"reaper","visibility_timeout":"5m0s","message":"reaper started"}
```

16.4 Container Resource Usage

Container	CPU	Memory	Net I/O	PIDs
scheduler_api	0.55%	59 MiB	551kB / 509kB	13
scheduler_worker	0.48%	58.5 MiB	12MB / 7.2MB	14
scheduler_postgres	0.64%	41.4 MiB	12.3MB / 20MB	16
scheduler_redis	0.97%	10.1 MiB	482kB / 265kB	7

scheduler_frontend	0.00%	38.5 MiB	93.4kB / 167kB	11
Total		207.5 MiB	All 5 services under 210 MB	

16.5 Docker Image Sizes

Image	Size	Notes
codility_ide-api	31.7 MB	Distroless, single static Go binary
codility_ide-worker	24.5 MB	Distroless, no shell
codility_ide-frontend	330 MB	Node.js runtime + Next.js standalone

Performance Evaluation

17.1 API Response Times

Endpoint	Method	Avg Latency	Notes
/health	GET	3.4 ms	Pings PostgreSQL connection pool
/api/v1/metrics/system	GET	3.4 ms	Aggregation across jobs + workers
/api/v1/orgs	GET	4.2 ms	Includes JWT validation overhead
/api/v1/auth/login	POST	~260 ms	bcrypt hash comparison dominates

17.2 Concurrent Load Test (50 requests, 10 threads)

Metric	Value
Min Latency	12.7 ms
Average	33.7 ms
P50 (Median)	27.3 ms
P95	70.4 ms
P99	109.8 ms
Throughput	29.7 req/s

17.3 Database Query Performance (EXPLAIN ANALYZE)

Query	Planning	Execution	Index Used
SKIP LOCKED dispatch (10 jobs, 959-row table)	1.990 ms	0.493 ms	idx_jobs_dispatch (partial)

17.4 Job Status Distribution

Status	Count	%	Bar
dead (DLQ)	459	47.9%	
pending	440	45.9%	
succeeded	50	5.2%	



High dead count reflects intentional testing of the retry engine and DLQ pathway.

Testing and Quality Assurance

Category	Method	Scope	Result
Unit: Backoff	go test ./pkg/retry/	Fixed, linear, exponential, jitter bounds, edge cases	Pass
Docker Build	docker compose up --build	All 5 services start healthy	Pass
Migration	Auto on API startup	384-line schema applied. Version=1, dirty=false	Pass
Auth: Login	curl POST /auth/login	Valid returns JWT. Invalid returns 401	Pass
Auth: Register	curl POST /auth/register	Created. Duplicate email returns conflict	Pass
Org CRUD	curl POST/GET/PATCH/DELETE	Create, list, update, delete. RBAC enforced	Pass
Job Enqueue	seed.py (500 jobs)	Bulk via API. All 500 inserted as pending	Pass
Idempotency	Duplicate POST with same key	Returns existing job (200) not duplicate (201)	Pass
Dispatch	Worker logs + DB	pending → running within 1 second	Pass
Execution	demo.sleep handler	Succeeds after 5s, correct duration_ms	Pass
Retry	Failing handler	Rescheduled with exponential delay	Pass
DLQ	Exceed max_retries	Transitions to dead. 459 dead confirmed	Pass
Heartbeat	workers table inspection	Updates every 5s. Two workers active	Pass
Graceful Shutdown	docker compose stop worker	In-flight job completed before exit	Pass
WebSocket	Browser DevTools	WS established. Events on state changes	Pass
Rate Limit	200 rapid-fire requests	First 100 pass. Then 429	Pass
Dashboard	Browser observation	Workers: 2, Pending: 440 live	Pass

Challenges and Solutions

Challenge	Root Cause	Solution
Frontend stale build	Next.js bakes production build into Docker image. Code changes invisible without rebuild.	Always use --build flag. Restructured workflow for Dockerized Next.js.
Dashboard all zeros	No auth guard in ClientLayout. API calls silently failed with 401.	Added useEffect redirect + Axios 401 interceptor.
Nested git repo	create-next-app creates its own .git directory.	Deleted frontend/.git, removed cached entry, restaged files.
Worker startup race	Worker connects before PostgreSQL is healthy.	Retry with exponential backoff in db.Connect(). depends_on with health condition.
CORS blocking	API :8080, frontend :3000. Missing origin config.	Fiber CORS with explicit AllowOrigins from env var.

Design Decisions and Trade-offs

Decision	Rationale	Trade-off
Go over Python/Node	Goroutines, compiled binary, tiny images (24-31 MB)	Steeper learning curve, verbose error handling
PostgreSQL as broker	SKIP LOCKED, ACID, complex filtering free	Lower throughput vs. in-memory brokers
Repository pattern	Testable with mocks, swappable backends	More boilerplate per entity
JWT over sessions	Stateless, horizontally scalable	Cannot instantly revoke (15 min window)
Dead as status field	Full history in one table, simple requeue	Needs archival at scale
UUID v4 PKs	No DB roundtrip for ID, no guessing	Slightly larger than integer PKs
Clean architecture	Each layer independently testable	More files and directories
Fiber over Gin	Fastest Go HTTP framework, Express-like	Non-standard net/http

Known Limitations

Limitation	Impact	Resolution
Single PostgreSQL instance	Single point of failure	Streaming replication + Patroni failover
No distributed tracing	Difficult end-to-end debugging	OpenTelemetry + Jaeger integration
No Kubernetes	Cannot auto-scale workers	Helm chart with HPA on jobs_pending metric
Limited frontend scope	Queue creation requires API calls	Add queue management + job explorer pages
No execution results	result JSONB column unused	Add handler return value contract
Single-region	No geographic distribution	Multi-region with queue affinity

Future Improvements

Improvement	Description
Kubernetes + Helm	HPA auto-scaling workers based on pending jobs metric
Prometheus + Grafana	/metrics endpoint, pre-built dashboards for latency and throughput
Queue sharding	Partition jobs table by queue_id for 100M+ row tables
Workflow DAG	parent_job_id FK exists; wire DAG scheduler for job dependencies
OpenAPI / Swagger	Auto-generate client SDKs in TypeScript, Python, Go

Project Statistics

7,866

LINES OF GO

1,101

LINES OF TS/CSS

413

LINES OF SQL

65

GO SOURCE FILES

11

FRONTEND FILES

5

DOCKER SERVICES

13

DATABASE TABLES

103

HTTP HANDLERS

6

PARTIAL INDICES

Lessons Learned

- **PostgreSQL is a capable job queue.** SKIP LOCKED eliminates the need for a separate broker for many workloads. Transactional guarantees are stronger than most brokers provide.
- **Partial indices are critical.** Without them, index size grows with total jobs. With them, only active jobs indexed -- sub-ms dispatch regardless of table size.
- **Graceful shutdown requires explicit design.** WaitGroup + context cancellation + pool draining ensures no work is lost on SIGTERM.
- **Jitter matters more than the backoff curve.** The +/-25% randomization eliminates thundering herd entirely.
- **Docker production builds differ from dev.** Next.js bakes the build into the image. Understanding multi-stage Dockerfile lifecycle was essential.
- **Clean architecture pays off in debugging.** Layered separation made it easy to isolate the dashboard zero-values issue to the frontend auth flow.

References

Technology	Reference
Go	https://go.dev/doc/
Fiber	https://docs.gofiber.io/
PostgreSQL 16	https://www.postgresql.org/docs/16/
SKIP LOCKED	https://www.postgresql.org/docs/16/sql-select.html#SQL-FOR-UPDATE-SHARE
Redis	https://redis.io/docs/
Next.js	https://nextjs.org/docs
TanStack Query	https://tanstack.com/query/latest/docs/
JWT (RFC 7519)	https://datatracker.ietf.org/doc/html/rfc7519
Docker Compose	https://docs.docker.com/compose/
golang-migrate	https://github.com/golang-migrate/migrate
pgx	https://github.com/jackc/pgx
bcrypt	https://pkg.go.dev/golang.org/x/crypto/bcrypt

Conclusion

This project implements a complete distributed job scheduling system demonstrating production-grade engineering:

- **Concurrency correctness:** SKIP LOCKED ensures exactly-once dispatch across any number of worker nodes
- **Reliability:** Configurable retries with jitter, heartbeat/reaper, graceful shutdown -- no silent job loss
- **Database design:** 13 tables, 8 ENUM types, partial indices, automatic triggers
- **API design:** RESTful hierarchy, consistent envelopes, RBAC, rate limiting, idempotency
- **Clean architecture:** Domain/service/repository separation, infrastructure-agnostic business logic
- **Observability:** Every attempt recorded. System metrics + WebSocket live updates

Repository: https://github.com/VermaVanshdeep/codity_assignment

Quick start:

```
git clone https://github.com/VermaVanshdeep/codity_assignment.git && cd
codity_assignment && cp .env.example .env && docker compose up --build -d && python3
seed.py
```